

Lab Evaluation 1

Roll: 2103064

1(a) Code Snippet:

```
%option noyywrap

%{
    #include "cal.tab.h"
}%

letter [A-Za-z]
digit [0-9]
us [_]
id ({us}|{letter})({us}|{digit}|{letter})*
number {digit}+

%%

"char" { printf("CHAR: %s\n",yytext); return CHAR;}
"malloc" { printf("MALLOC: %s\n",yytext); return MALLOC;}
"sizeof" { printf("SIZEOF: %s\n",yytext); return SIZEOF;}
"for" { printf("FOR: %s\n",yytext); return FOR;}
"free" { printf("FREE: %s\n",yytext); return FREE;}
"***" { printf("POINTER: %s\n",yytext); return POINTER;}
"++" { printf("INC: %s\n", yytext); return INC;}
"(" { printf("LP: %s\n", yytext); return LP;}
")" { printf("RP: %s\n", yytext); return RP;}
"<" { printf("LT: %s\n", yytext); return LT;}
"=" { printf("EQUAL: %s\n", yytext); return EQUAL;}
"[" { printf("LS: %s\n", yytext); return LS;}
"]" { printf("RS: %s\n", yytext); return RS;}
";" { printf("SEMI: %s\n", yytext); return SEMI;}
"*" { printf("MUL: %s\n", yytext); return MUL;}

{id} { printf("ID: %s\n", yytext); return ID; }

{number} { printf("NUM: %s\n", yytext); return NUM; }

[ \t\n]+ { }

%%
```

1(a) Output.

```
CHAR: char
POINTER: **
ID: names
EQUAL: =
LP: (
CHAR: char
POINTER: **
RP: )
MALLOC: malloc
LP: (
ID: n
MUL: *
SIZEOF: sizeof
LP: (
CHAR: char
MUL: *
RP: )
RP: )
SEMI: ;
FOR: for
LP: (
ID: i
EQUAL: =
NUM: 0
SEMI: ;
ID: i
LT: <
ID: n
SEMI: ;
ID: i
INC: ++
RP: )
ID: names
LS: [
ID: i
RS: ]
EQUAL: =
LP: (
CHAR: char
MUL: *
RP: )
MALLOC: malloc
LP: (
ID: size
MUL: *
SIZEOF: sizeof
LP: (
CHAR: char
RP: )
```

```
RP: )
SEMI: ;
FOR: for
LP: (
ID: i
EQUAL: =
NUM: 0
SEMI: ;
ID: i
LT: <
ID: n
SEMI: ;
ID: i
INC: ++
RP: )
FREE: free
LP: (
ID: names
LS: [
ID: i
RS: ]
RP: )
SEMI: ;
FREE: free
LP: (
ID: names
RP: )
SEMI: ;
```

1(b) Code Snippet:

```
%{  
  
    #include <stdio.h>  
    // #include <stdlib.h>  
  
    void yyerror(const char *s);  
    int yylex();  
  
}%  
  
%token CHAR MALLOC SIZEOF FOR FREE POINTER INC LP RP LT EQUAL LS RS SEMI MUL ID  
NUM  
%start cal  
  
%%  
cal: statements  
    ;  
  
statements: statement statements  
    |  
    ;  
  
statement: declaration  
    | for_loop  
    | value_assign  
    | free1  
    | free2  
  
    ;  
/* char ** names = ( char ** ) malloc ( n * sizeof ( char * ) ) ; */  
declaration:  
    CHAR POINTER ID EQUAL LP CHAR POINTER RP MALLOC LP ID MUL SIZEOF LP CHAR  
MUL RP RP SEMI  
    ;  
/* for ( i = 0 ; i < n ; i++ ) */  
for_loop:  
    FOR LP ID EQUAL NUM SEMI ID LT ID SEMI ID INC RP  
    ;
```

```

/* names [ i ] = ( char * ) malloc ( size * sizeof ( char ) ) ; */
value_assign:
    ID LS ID RS EQUAL LP CHAR MUL RP MALLOC LP ID MUL SIZEOF LP CHAR RP RP
SEMI
    ;

/* free ( names [ i ] ) ; */
free1:
    FREE LP ID LS ID RS RP SEMI
    ;

/* free ( names ) ; */
free2:
    FREE LP ID RP SEMI
    ;

%%

void yyerror(const char *s)
{
    fprintf(stderr, "Syntax Error: %s\n", s);
}

int main()
{
    if (yyparse() == 0) {
        printf("Statements parsed successfully!\n");
    }
    return 0;
}

```

1(b) Output.

Statements parsed successfully!

Makefile:

```
main: cal.y cal.l  
    bison -d cal.y  
    flex cal.l  
    gcc cal.tab.c lex.yy.c  
    a < input.txt > output.txt
```